

Training a Code LLM from Scratch • Stage 2

SFT, the Data-Cleaning Detours, and a Late Lesson

SFT alignment • repeated data-quality pitfalls • the ceiling of a 0.6B model • KV-cache/RoPE debug • and the "train-from-scratch vs strong-base" course correction

Outline

- **0.** Recap: from Stage 1 (pretrain, writes C++) to Stage 2 (SFT, teach solving)
- **1.** What SFT is: warm start, problem-solution format, loss only on the answer
- **2.** Batching variable-length samples: padding + one loss mask suffices (causal's natural isolation)
- **3.** The repeated data-cleaning pitfalls: mixed-in Python, over-strict compilation, the heuristic trade-off
- **4.** Checks after the raw model finished: token coverage / single-byte fallback / embedding sparse-update verification / generation test
- **5.** The KV-cache RoPE position bug: training right, inference broken (with quantitative analysis)
- **6.** Validation: seen vs unseen problems — SFT memorizes, doesn't generalize
- **7.** The late lesson: the 0.6B ceiling (the best 0.5B scores only ~17% on HumanEval)
- **8.** Course correction: training a small model from scratch vs a ready strong base + post-train
- **9.** Capability breakdown by stage: pretraining is the foundation, SFT/RL are the finishing touches

0. Recap: from "writes C++" to "teach solving"

Stage 1 hand-wrote a Transformer from scratch (random init), pretrained a 0.6B model on 14B C++ tokens, and it **writes fluent C++** (loss 0.77, ppl 2.1). Stage 2 is SFT: on top of pretraining, using "problem→solution" data, teach it to **"read a problem, write a solution"**.

Full chain: **pretrain (writes) → SFT (align: read-and-solve) → RL (generalize: actually solve correctly)**. This piece covers SFT, the pitfalls along the way, and the fundamental problem finally seen clearly — the 0.6B ceiling.

1. SFT: warm start + problem-solution format + answer-only loss

SFT keeps the model architecture unchanged, changing three things:

- **warm start:** load the pretrained checkpoint (writes C++), continue training — not from scratch.
- **data format:** "problem + solution" pairs with a separator (`### Problem: ... ### Solution: ...`), `<eos>` at the end of the solution (learn when to stop).
- **loss only on the answer (solution):** the problem part's label is set to -100 (not counted, just a condition); the model learns "given a problem, generate a solution".

The mechanism is still next-token + CE (same as pretraining), only the data is structured (problem+solution) and loss is only on the answer. SFT loss starts from the pretrained 0.77 (warm start, already writes) and drops to ~0.5 (learns to read-and-solve).

SFT is astonishingly fast (~1 hour): small data (~10M tokens, a fraction of a thousandth of pretraining's 14B). Because SFT is "alignment" (activating capabilities the pretraining already has), not "learning capability" — small data suffices. A paper (LeetCodeDataset) found **2.6K high-quality SFT ≈ 110K samples**, confirming "SFT quality > quantity".

2. Batching variable-length samples: one mask suffices

Each SFT sample is a "problem+solution" unit, of varying length. Batching requires padding to the longest. A natural question: does padding need its own mask?

Conclusion: one loss mask suffices (problem=0, solution=1, padding=0). No separate attention padding mask is needed, because:

- padding is added at the **end**;
- the model has a **causal mask** (each token only sees what's before it);
- real tokens (problem+solution) are all before the padding → blocked by causal, naturally can't see the trailing padding;
- padding's own output has loss mask=0, so it doesn't matter.

So padding-at-end + causal = natural isolation, one loss mask does it. (Padding mask and causal mask are essentially the same: both set "what shouldn't be seen" to 0.)

3. The repeated data-cleaning pitfalls

SFT quality depends almost entirely on data quality. This stretch is repeated trial-and-error — each step forced back by "the generations are wrong".

Pitfall 1: mixed-in Python (CodeContests' language label is wrong)

The first SFT dataset mixed Magicoder (labeled cpp) + CodeContests (filtered by language==3, i.e. C++). After training: problem 1 (find max) was right, **but factorial spat out Python (`for i in range(1,n)`), and others repeated endlessly.**

Tracing back: CodeContests' `language==3` **is mislabeled** — many marked as C++ are actually Python (`from collections import deque` , `n=int(input())`). The data poisoned the model.

Pitfall 2: compilation check too strict (misfiltering real C++)

Switched to g++ compile verification (compiles = real C++). Result: **only 14% passed (1745/12827)**, and CodeContests was almost entirely wiped out.

Looking at the failures revealed two kinds: ① CodeContests indeed has lots of Python (should be filtered, failing to compile is correct) ② **but Magicoder is real C++, just depending on special libraries (`#include <GL/glew.h>`) it fails to compile and got misfiltered.**

Compilation is too strict: fragments/library-dependent real C++ get filtered out too.

Trade-off: heuristics (exclude Python + confirm C++ features)

Finally used **heuristic filtering** (not strict compilation): exclude Python features (`def/print/range/input`) + confirm C++ features (`#include/cout/std/{}/;` ≥ 3). This both removes Python contamination and keeps library-dependent real C++.

Added a clean source **AmareshHebbar/leetcode-codegen-cpp** (from the doocs community solution repo, human-written correct solutions, messages format: user=problem, assistant=solution).

Final: **Magicoder 3567 + AmareshHebbar 2901 = 6468 clean C++ problem-solutions.**

Cleaning lessons: ① Don't trust metadata's language labels (CodeContests' language is wrong) — verify the code content. ② Compilation is most accurate but too strict (fragments/library-dependent real code misfiltered); heuristics are a pragmatic trade-off. ③ Data quality > quantity: the dirty 12773 trained to only 1 correct; the clean 6468 clearly better — but still limited by the next section's ceiling.

4. Checks after the raw model finished

After pretraining the raw model (loss 0.77), two kinds of checks: ① static diagnostics (what the model learned internally) ② generation test (does it actually work). Both had important findings.

4.1 Token coverage diagnostic: which tokens got trained

Scanning all 14B tokens, counting each token's frequency (= number of BP updates), to see vocab coverage:

Category	Count	Share
touched (appeared)	30245	94.5%
never (never appeared)	1761	5.5%
high-freq ($\geq 10k$)	27594	86%

What are the 1761 never tokens: ① special tokens (`<pad>/<bos>/<unk>` , pretraining used only eos as separator) ② rare high bytes (`À Á Õ õ ÷ ...ÿ` , i.e. 0x80-0xFF, almost never in C++) ③ **surprise: space-prefixed protected keywords (`Ġif Ġreturn Ġconst Ġstd`) are also never** — meaning in code these keywords more often appear in another form (no leading space).

4.2 Single-byte fallback analysis

In byte-level BPE, "whatever isn't merged falls back to single bytes". Stats: 367 single-byte tokens, 13 never (exactly the rare high bytes). **Single bytes make up 42.3% of token occurrences** — lots of content (symbols/spaces/single letters) exists as single bytes. This confirms: **not every single byte gets touched** (common chars high-freq, rare bytes never), and "anything unseen ultimately falls back to single bytes" (the fallback layer).

4.3 Embedding sparse-update verification (first principles)

BP updates embeddings sparsely: only rows of tokens that appeared have gradients and get updated; unseen ones have gradient exactly 0 and their embedding doesn't change (mathematically inevitable). Verification — bucketing embedding norms by frequency:

frequency	token count	avg norm
never(0)	1761	0.751
1~100	75	0.755
100~10k	2576	0.926
10k~1M	26283	1.023

Frequency-norm perfectly monotonic: never's norm $0.751 \approx$ the random-init theoretical value ($0.02 \times \sqrt{d} \approx 0.716$); higher frequency, more deviation from random (high-freq 1.02). **This quantitatively verifies "trained only if it appeared"** — unseen embeddings are still

random-init values.

The neat part: judging never doesn't require storing the pre-training initial embedding (not stored) — because init is a known distribution $N(0, 0.02^2)$; never's embedding must still follow it (0 updates, unmoved); just check "still $\approx$ random distribution". Use the "known initial distribution" in place of the "unstored initial values".

4.4 Generation test: discovering the KV-cache RoPE bug

Static diagnostics showed the model trained well (loss 0.77, coverage 94.5%, embeddings normal), but running generation (KV-cache) produced **endless repetition** — exposing the next section's bug.

5. The KV-cache RoPE position bug: training right, inference broken

After pretraining, loss 0.77 (great), but KV-cache generation was **all repetition** (`return return return...`). Good loss + broken generation = not a model problem, but a generation-code (KV-cache) bug.

Root cause: increment generation used position 0 for RoPE

Training (full-sequence forward, no KV-cache): RoPE applied correctly per position → positions right → loss good.

Inference (KV-cache incremental, emitting one token at a time): `apply_rope` uses `cos/sin[:seq]`, and at increment `seq=1` → it takes **position 0**'s rotation → the new token (actually the n-th) is treated as "position 0" → all positions wrong → broken.

The training path (full sequence, positions right) and inference path (KV-cache, position bug) are two paths — training right, inference wrong, hence "trained well but generation broken".

Quantitative: why wrong positions cause repetition

Let `head_dim=2`, `K` originally all `[1,0]`.

- **positions right:** `K` rotated by positions `0/1/2` → `[1,0]/[0.54,0.84]/[-0.42,0.91]`; `Q(pos3) · K` = `-0.99/-0.41/0.55` (**different**) → softmax weights differentiated (`0.10/0.24/0.66`) → attends to different positions.
- **positions all wrong at 0:** `K` unrotated = all `[1,0]`; `Q · K` = `1/1/1` (**all identical**) → uniform weights (`0.33/0.33/0.33`) → same attention to every position → **identical output each step → repetition**.

Wrong positions → rotation all 0 → `Q · K` loses differentiation → weights degenerate to uniform → repetition.

Fix: add an `offset` parameter to `apply_rope`, pass the current position at increment (`offset=n`), take `cos/sin[offset:offset+seq]` = the true position's rotation. After the fix, KV-cache generation is normal (no repetition).

Lesson: "training/inference path mismatch" is a classic LLM bug — training uses full sequences (positions naturally right), inference uses KV-cache increments (positions must be passed correctly by hand). Good loss doesn't mean correct generation; both paths must be validated.

6. Validation: seen vs unseen problems

After SFT (clean 6468 data), validation of two kinds: common training-set problem types vs deliberately-crafted unseen problems. The contrast exposes SFT's nature.

Seen common problems: partly correct

"array sum" → fully correct (read input + accumulate + output); "factorial" → structure right (class + recursion), just the base case wrong (`return 0` should be `return 1`); "longest substring / palindrome" → structure looks right (two-pointer/sliding-window shape) but logic wrong.

Basically writes plausible C++ structure, simple problems logically correct, complex problems structurally right but logically wrong. A clear improvement over dirty data's "1 correct + Python spillover".

Unseen problems: poor generalization (mismatched)

Deliberately-crafted new problems (unlikely in the training set):

- "return second largest element" → wrote a **segment tree** (irrelevant)
- "filter words longer than 5" → wrote **merge sort** (irrelevant)
- "count primes below n" → wrote **counting factors** (logic wrong)
- "reverse word order in a sentence" → wrote a positive/negative reorder (irrelevant)

6 problems basically all wrong, with lots of **mismatched answers** — the model didn't read the problem, it grabbed a few words and applied a seen pattern.

The stark seen-vs-unseen contrast = SFT memorizes problem-type forms, doesn't generalize. A deeper issue: the many "mismatched answers" mean the model **can't read the problem** (weak natural-language understanding) — this isn't dirty data (the LeetCode data is correct), it's model capability: **pure-C++ pretraining (almost no natural language) → weak problem understanding → grab words, apply patterns → mismatch.**

7. The late lesson: the 0.6B ceiling

By this point in validation, a problem that should have been faced from the start surfaced: **at the 0.6B scale, the ceiling for solving ability is extremely low.**

Model	Pretrain data	HumanEval pass@1
Qwen2-0.5B (best 0.5B)	5.5T tokens	~17%
Qwen2.5-Coder-1.5B	5.5T tokens	~44%
Qwen2.5-Coder-7B	5.5T tokens	~85%
This project 0.6B	14B tokens (~400× less)	far below 17%

The best 0.5B (trained on 5.5T tokens) scores only 17% on HumanEval (164 problems, only ~28 solved). This project's 0.6B used 14B tokens (400× less), so the ceiling is even lower — "mismatch, can't read problems" is inevitable, not an SFT/RL mistake.

The most important lesson: the foundation of capability is pretraining. 0.6B + 14B tokens of pretraining is too weak (weak reading, weak logic), and SFT/RL can't save it. And — **"if the policy is too poor, hoping RL saves it is very hard"**: RL optimizes by trial-and-error on the existing policy; if the policy is so poor that samples are all wrong (no positive samples), RL can't learn (exactly like "if exploration never hits a positive sample, it can't learn" in CarRacing). RL amplifies a good policy; it can't save a bad one. This ceiling should have been faced when choosing 0.6B: if the goal is "learn the mechanisms, walk the full pipeline", 0.6B is fine (cheap, complete); if the goal is "real solving ability", 0.6B was insufficient from the start, and a ready strong base should have been used.

8. Course correction: from scratch vs strong base

Training a small model from scratch (what this project did): weak pretraining foundation (14B tokens), low ceiling. Value is in **learning the mechanisms** (understanding pretraining/SFT/RoPE/KV-cache/GRPO at the bottom level), not in solving ability.

Ready strong base + post-train (the correct route): download Qwen2.5-Coder / Gemma etc. (already trained on trillions of tokens, HumanEval 44-85%), **the pretraining step is "freeloaded"** (the hundreds of thousands of dollars a big lab burned), and you only do SFT + RL (small cost ~\$1000). On a strong base: SFT is effective, RL is meaningful (a good policy has positive samples, big gains, e.g. DeepSeek-R1 raised AIME from 15% to 78% via RL).

The project's real value: hand-writing the full pretrain→SFT pipeline from scratch, deriving $CE = \log|\text{vocab}|$, perplexity=effective-candidate-count, RoPE rotation/KV-cache, SFT's mask loss, GRPO principles, and thoroughly hitting the pitfalls of data cleaning / memory / speed /

training-inference mismatch. **This bottom-level understanding is the hard foundation for "using a ready base + post-train"** — knowing what each step does, rather than black-box API calls.

9. Capability breakdown by stage: pretraining is the foundation

Stage	Contribution (rough)	Role	This project's pitfall
Pretraining	~80-90%	writes code / reads problems / algorithm knowledge (foundation)	14B tokens too weak (root cause)
SFT	~5-10%	aligns format (immediate but memorizing)	repeated data cleaning; poor generalization
RL	~5-15%	generalize+refine (only effective with a good base)	0.6B poor policy, RL can't save

Key insight: SFT is immediate (alignment), easily leading to "SFT is the key, RL is limited". But that's **an illusion on a weak base** — on a strong base + good SFT, RL gains are huge (R1). The foundation is always pretraining; SFT aligns, RL generalizes, both finishing touches on the foundation. This project's 0.6B is weak because the foundation (pretraining) is weak, not SFT/RL's fault.

Next step

Abandon "train 0.6B from scratch to solve LeetCode" (ceiling inevitable), pivot to **ready strong base + post-train (SFT+RL)**. The goal lands on the **Google Gemma 4 Developer Agent Competition** — post-train Gemma 4 into an autonomous coding agent (SFT+RL, offline, agentic), exactly where this project's accumulation (SFT/RL code, data cleaning, bottom-level math) applies, and the route is correct (strong base + post-train).

— End of Stage 2. Lesson: pretraining is the foundation, the 0.6B ceiling is inevitable; pivot to strong base + post-train. —